Information Security Lab Week 12

Web Application Attacks – SQL Injection (SQLi) & Cross-Site Scripting (XSS)

Instructor Notes (Detailed Explanation)

Lab Objectives

After completing this lab, students should be able to:

- Understand the concept of SQL Injection (SQLi).
- Understand how authentication bypass works.
- Understand the purpose of Boolean-based SQL Injection.
- Understand the concept of Cross-Site Scripting (XSS).
- Differentiate between Reflected XSS and Stored XSS.
- Analyze attack impacts and discuss prevention techniques.

Introduction to Web Application Security

Before starting the experiments, explain the following concept to students:

Most websites interact with a database.

For example, when a user enters:

- Username: Ali
- Password: 123

The application may execute a query similar to:

```
SELECT * FROM users
WHERE username='Ali'
AND password='123';
```

If a matching record exists, the user is logged in.

If the application does not properly validate user input, attackers may manipulate the query and change its behavior.

This type of attack is called **SQL Injection (SQLi)**.

Similarly, if a website displays user input without proper filtering, attackers can inject JavaScript code into web pages.

This attack is called **Cross-Site Scripting (XSS)**.

EXPERIMENT 1

SQL Injection Authentication Bypass

Theory

Authentication bypass is one of the most common SQL Injection attacks.

Consider the following vulnerable query:

```
SELECT * FROM users
WHERE username='input'
AND password='input';
```

The application expects a valid username and password.

However, if user input is directly inserted into the SQL query without validation, an attacker may alter the query logic.

Conceptual Example

A malicious input can change the WHERE condition so that it always evaluates as TRUE.

As a result, the application may authenticate a user without verifying the correct password.

How SQL Injection Works

Normally:

Username = Ali
Password = 123

The application searches for a matching user.

However, malicious input may alter the query structure and force the database to return records that should not be accessible.

This allows attackers to bypass authentication controls.

Expected Learning Outcome

Students should understand:

- Why user input should never be directly inserted into SQL queries.
- How SQL Injection manipulates query logic.
- Why authentication bypass occurs.
- The importance of input validation and parameterized queries.

Task 1 – Authentication Bypass Report

Students must submit:

1. Screenshot of the login page.
2. Screenshot of the application behavior after testing.
3. Explanation of:
    - What SQL Injection is.
    - How the attack modified query behavior.
    - Why authentication controls failed.
4. Prevention techniques.

EXPERIMENT 2

Extracting Database Information via SQL Injection

Theory

After discovering a SQL Injection vulnerability, attackers often attempt to gather information about the database.

Their objectives may include:

- Discovering table names.
- Identifying column names.
- Accessing user information.
- Accessing database metadata.

This phase is called **information gathering** or **database enumeration**.

Understanding Error-Based Discovery

Attackers frequently begin by testing how the application handles unexpected input.

Database error messages may reveal:

- Database type.
- Table structure.
- Query behavior.
- Number of columns returned by a query.

Even small error messages can provide valuable information.

Understanding Column Enumeration

Before combining data from different queries, attackers must understand how many columns are returned by the original query.

Determining the number of columns is an important step because database operations often require matching structures.

Understanding UNION-Based SQL Injection

SQL provides the UNION operator.

Purpose:

Combine results from two separate SELECT statements.

Example:

SELECT name,email FROM customers

UNION

SELECT username,password FROM users;

If used improperly, attackers may attempt to retrieve information from tables that were never intended to be exposed.

Expected Learning Outcome

Students should understand:

- Why database errors can leak information.
- Why attackers identify column counts.
- How database information disclosure occurs.
- Why secure query design is important.

Task 2 – SQL Data Extraction Report

Students must create a table:

User ID Username Password/Hash

Students must also answer:

Question 1

Why can UNION-based SQL Injection expose sensitive data?

Question 2

How can developers prevent unauthorized database access?

Attach

- Screenshots of observed error messages.
- Screenshots demonstrating database information disclosure.
- Analysis of findings.

EXPERIMENT 3

Boolean-Based SQL Injection

Theory

Many modern applications hide database error messages.

When direct error messages are unavailable, attackers may use Boolean logic.

Boolean logic has two possible outcomes:

TRUE
FALSE

The attacker sends different conditions and observes how the application's response changes.

How Boolean SQL Injection Works

Suppose a condition evaluates to:

TRUE

The page may display normal content.

If the condition evaluates to:

FALSE

The page may display:

- No results.
- Different content.
- A different page length.
- Different response behavior.

By comparing responses, attackers may infer information about the database.

Why Boolean SQL Injection is Dangerous

Even when error messages are hidden:

- Database information can still be inferred.
- Application behavior can leak information.
- Attackers can gradually discover sensitive data.

Expected Learning Outcome

Students should understand:

- The role of TRUE and FALSE conditions.
- How response comparison works.
- Why hidden errors do not completely eliminate SQL Injection risks.

Task 3 – Boolean Logic Report

Answer the following questions:

Question 1

What differences did you observe between TRUE and FALSE conditions?

Question 2

What does the application's behavior reveal about backend query execution?

Question 3

Why is Boolean-based SQL Injection useful when error messages are hidden?

Attach

- Screenshots of observations.
- Explanation of results.

EXPERIMENT 4

Reflected Cross-Site Scripting (XSS)

Theory

Cross-Site Scripting (XSS) occurs when a web application displays user input without proper sanitization.

Instead of displaying the input as plain text, the browser may interpret it as executable code.

As a result, JavaScript code supplied by a user may execute inside another user's browser.

Reflected XSS

Reflected XSS occurs when:

1. User submits input.
2. Application immediately displays the input.
3. Browser executes the injected script.

The malicious code is not stored permanently.

Possible Consequences

An attacker may:

- Steal session cookies.
- Redirect users to fake websites.
- Modify page content.
- Capture user actions.
- Impersonate users.

Viewing Source Code

Students should inspect the page source and identify where their input appears inside the HTML document.

This helps them understand how the browser interprets user-supplied content.

Expected Learning Outcome

Students should understand:

- How browsers process HTML and JavaScript.
- Why input validation is important.

- The impact of client-side code execution.

Task 4 – XSS Demonstration Report

Students must provide:

1. Screenshot showing script execution.
2. Screenshot of page source.
3. Explanation of:
    - What Reflected XSS is.
    - Why the browser executed the script.
    - Potential real-world consequences.

EXPERIMENT 5

Stored (Persistent) XSS

Theory

Stored XSS is more dangerous than Reflected XSS.

In Stored XSS:

1. Malicious input is submitted.
2. The application saves the input.
3. The input is stored in a database.
4. Every future visitor receives the malicious content.

The attack remains active until the content is removed.

Why Stored XSS is Dangerous

Unlike Reflected XSS:

- The attacker does not need to resend the payload.
- Every user who visits the page may be affected.
- The attack can spread to many users automatically.

Real-World Impact

Stored XSS may lead to:

- Session hijacking.

- Account takeover.
- Data theft.
- Website defacement.
- Distribution of malicious content.

Expected Learning Outcome

Students should understand:

- The difference between temporary and persistent attacks.
- Why database storage increases risk.
- How web applications should sanitize user input.

Task 5 – Stored XSS Report

Answer the following questions:

Question 1

Why is Stored XSS considered more dangerous than Reflected XSS?

Question 2

Describe two possible real-world consequences of Stored XSS.

Question 3

What security measures can prevent Stored XSS attacks?

Attach

- Screenshots of observations.
- Evidence of stored script execution.
- Detailed explanation.

Class Participation Task (10 Marks)

One-Page Report

Topic

“Compare SQL Injection and Cross-Site Scripting (XSS) from both an Attacker’s and Defender’s Perspective.”

Students must discuss:

1. Attack Vector

How each attack is performed.

2. Target

What the attacker is trying to compromise.

3. Impact

Potential consequences.

4. Difficulty Level

Complexity of performing the attack.

5. Prevention Techniques

Security controls used to defend against the attack.